Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №3
По дисциплине: «КиСБ»
Тема: «Блочные шифры и классические режимы шифрования»

Выполнил:
Студент 4-го курса
Группы АС-65
Колбашко А.В.
Проверил:
Хацкевич А.С.

Брест 2026

Цель работы: программная реализация симметричного блочного шифра (AES-128 или СТБ 34.101.31 «БелТ») и классических режимов работы (ECB, CBC, CFB, OFB, CTR) с поддержкой алгоритма дополнения блоков (Padding) Задание.
В лабораторной работе требуется реализовать указанный блочный шифр и продемонстрировать его работу в двух режимах шифрования: обязательном базовом режиме ECB и дополнительном режиме согласно варианту. Вариант 9:

9	AES-128	ECB + CBC	Строгая проверка *IV* и цепочек блоков. Демонстрация уязвимости ECB на графическом файле.

AES-128 шифрует блок 16 байт ключом 16 байт. Из ключа создаются 11 раундовых ключей при помощи `RotWord`, `SubWord`, `Rcon` и XOR. После начального `AddRoundKey` выполняются девять раундов `SubBytes → ShiftRows → MixColumns → AddRoundKey`. Последний, десятый раунд не содержит `MixColumns`. При расшифровании применяются обратные операции. S-box и `Rcon` заданы готовыми константами.

AES128` отдельно шифрует и расшифровывает один блок (`encrypt_block`, `decrypt_block`). Функции `encrypt_ecb`, `decrypt_ecb`, `encrypt_cbc`, `decrypt_cbc` обрабатывают файлы поблочно; `add_padding`, `remove_padding` реализуют PKCS#7. `main` читает выбранный файл, шифрует его, ждёт Enter, повторно читает шифротекст с диска и расшифровывает его.

Класс алгоритма AES:

```
class AES128 {
    unsigned char keys[176];
    unsigned char inverse_s[256];

    void add_key(unsigned char state[16], int round) const {
        for (int i = 0; i < 16; i++) state[i] ^= keys[round * 16 + i];
    }

    void sub_bytes(unsigned char state[16], bool inverse) const {
        for (int i = 0; i < 16; i++) state[i] = inverse ? inverse_s[state[i]] :
S[state[i]];
    }

    void shift_rows(unsigned char state[16], bool inverse) const {
        unsigned char copy[16];
        for (int i = 0; i < 16; i++) copy[i] = state[i];
        for (int row = 0; row < 4; row++) {
            for (int col = 0; col < 4; col++) {
                int from = inverse ? (col - row + 4) % 4 : (col + row) % 4;
                state[4 * col + row] = copy[4 * from + row];
            }
        }
    }

    void mix_columns(unsigned char state[16], bool inverse) const {
```

```
        for (int col = 0; col < 4; col++) {
            unsigned char a = state[4 * col];
            unsigned char b = state[4 * col + 1];
            unsigned char c = state[4 * col + 2];
            unsigned char d = state[4 * col + 3];
            if (inverse) {
                state[4 * col] = gf(a, 14) ^ gf(b, 11) ^ gf(c, 13) ^ gf(d, 9);
                state[4 * col + 1] = gf(a, 9) ^ gf(b, 14) ^ gf(c, 11) ^ gf(d, 13);
                state[4 * col + 2] = gf(a, 13) ^ gf(b, 9) ^ gf(c, 14) ^ gf(d, 11);
                state[4 * col + 3] = gf(a, 11) ^ gf(b, 13) ^ gf(c, 9) ^ gf(d, 14);
            }
            else {
                state[4 * col] = gf(a, 2) ^ gf(b, 3) ^ c ^ d;
                state[4 * col + 1] = a ^ gf(b, 2) ^ gf(c, 3) ^ d;
                state[4 * col + 2] = a ^ b ^ gf(c, 2) ^ gf(d, 3);
                state[4 * col + 3] = gf(a, 3) ^ b ^ c ^ gf(d, 2);
            }
        }
    }

public:
    AES128(const unsigned char key[16]) {
        for (int i = 0; i < 256; i++) inverse_s[S[i]] = i;
        for (int i = 0; i < 16; i++) keys[i] = key[i];
        for (int word = 4; word < 44; word++) {
            unsigned char temp[4];
            for (int j = 0; j < 4; j++) temp[j] = keys[4 * (word - 1) + j];
            if (word % 4 == 0) {
                unsigned char first = temp[0];
                temp[0] = S[temp[1]] ^ RCON[word / 4 - 1];
                temp[1] = S[temp[2]];
                temp[2] = S[temp[3]];
                temp[3] = S[first];
            }
            for (int j = 0; j < 4; j++) keys[4 * word + j] = keys[4 * (word - 4) +
j] ^ temp[j];
        }
    }

    void encrypt_block(const unsigned char input[16], unsigned char output[16])
const {
        unsigned char state[16];
        for (int i = 0; i < 16; i++) state[i] = input[i];
        add_key(state, 0);
        for (int round = 1; round <= 9; round++) {
            sub_bytes(state, false);
            shift_rows(state, false);
            mix_columns(state, false);
            add_key(state, round);
        }
        sub_bytes(state, false);
        shift_rows(state, false);
        add_key(state, 10);
        for (int i = 0; i < 16; i++) output[i] = state[i];
    }

    void decrypt_block(const unsigned char input[16], unsigned char output[16])
const {
        unsigned char state[16];
        for (int i = 0; i < 16; i++) state[i] = input[i];
        add_key(state, 10);
        for (int round = 9; round >= 1; round--) {
            shift_rows(state, true);
            sub_bytes(state, true);
            add_key(state, round);
            mix_columns(state, true);
```

```
        }
        shift_rows(state, true);
        sub_bytes(state, true);
        add_key(state, 0);
        for (int i = 0; i < 16; i++) output[i] = state[i];
    }
};
```

Зашифрование блока AES:

```
add_key(state, 0);
for (int round = 1; round <= 9; round++) {
    sub_bytes(state, false);
    shift_rows(state, false);
    mix_columns(state, false);
    add_key(state, round);
}
sub_bytes(state, false);
shift_rows(state, false);
add_key(state, 10);
```

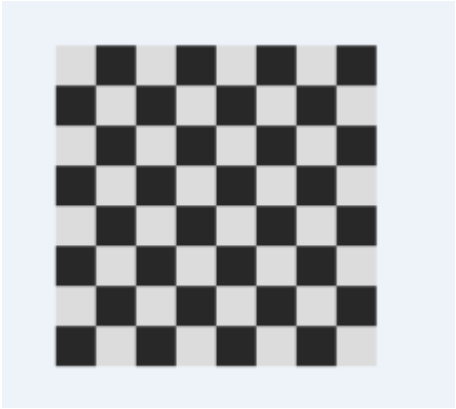
Дополнение и его проверка:

```
int count = 16 - data.size() % 16;
for (int i = 0; i < count; i++)
data.push_back(count);

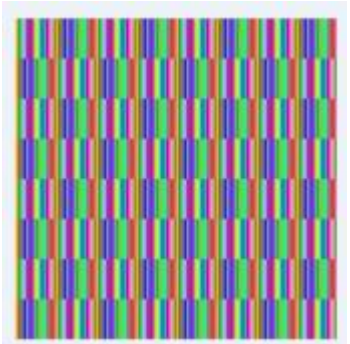
int count = data.back();
if (count < 1 || count > 16) return false;
for (int i = 0; i < count; i++)
    if (data[data.size() - 1 - i] != count) return
false;
data.resize(data.size() - count);
```

Результат работы программы

Исходное изображение:



Зашифрованное изображение в режиме ECB и CBC:



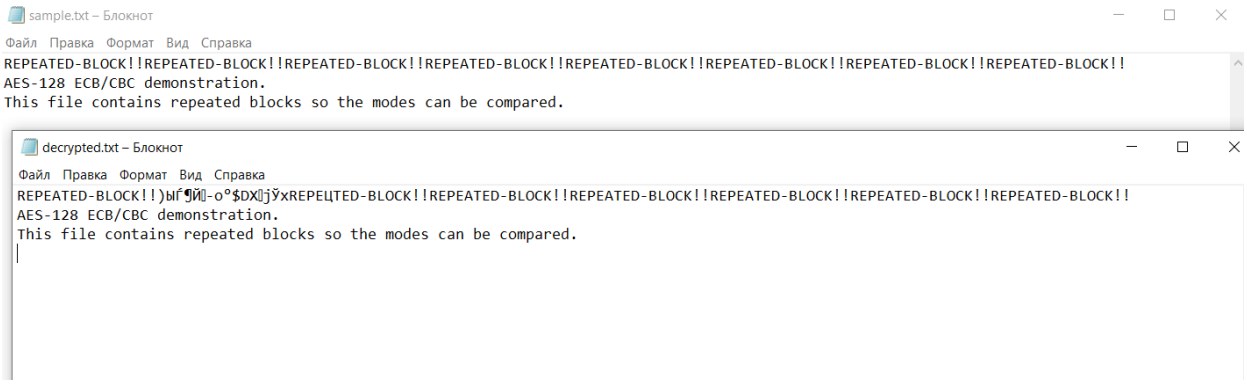


(Как видно из-за того что ECB шифрует блоки независимо без участия вектора инициализации и связи между блоками то в повторяющихся исходных паттернах получаются повторяющиеся шифрованные паттерны, которые позволяют нарушают криптостойкость)

Дешифрованное изображение обоих режимов:



Также был выполнен эксперимент для режима CBC если в зашифрованном файле испортить например один бит какого-либо блока то в расшифрованном сообщении будет испорчен весь этот блок и один бит в следующем блоке, но все последующие блоки остаются правильными.



Вывод : Алгоритм симметричного шифрования AES-128 позволяет с помощью секретного ключа шифровать блоки данных в различных режимах их работы для последующей безопасной передачи в канале связи.